# WebClient Computing

Javascript Basics

Department of Computer Science

Kyungyong Lee

# For Loop in Javascript

- for (initialize; condition; update)
  - The initialize code always executes
  - If the condition is not true, the body does not execute
  - After the body execution, update is executed
  - We can omit any of three statements where necessary
  - break and continue in for loop
    - break – always escape the current for loop
    - continue – perform update and check condition then execute

# For Loop Example

```
arr1 with while loop: 1
arr1 with while loop: 3
arr1 with while loop: 5
arr1 with while loop: 7
arr1 with while loop: 9
arr1 with for loop: 1
arr1 with for loop: 3
arr1 with for loop: 5
arr1 with for loop: 7
arr1 with for loop: 9
print even number: 2
print even number: 4
print even number: 6
print even number: 8
```

```javascript
var arr1 = [1,3,5,7,9];
var index = 0;

while (arr1[index] != undefined) {
  console.log("arr1 with while loop: " + arr1[index]);
  index++;
}

for(var index = 0; arr1[index] != undefined; index++) {
  console.log("arr1 with for loop: " + arr1[index]);
}

var arr2 = [1,2,3,4,5,6,7,8,9];
for(var index = 0; ; index++) {
  if (arr2[index] == undefined) {
    break;
  }

  if (arr2[index] % 2 != 0) {
    continue;
  } else {
    console.log("print even number: " + arr2[index])
  }
}
```

# Iterate For Loop Using in

- Iterate through object keys using in operator
  - for (var key in object)
    - key is the key of the object

```
var input = {name: "kyungyong", work: "KMU", age: 100};

for(var i in input) {
  console.log("key is: " + i + " value is " + input[i]);
}
```

```
key is: name value is kyungyong
key is: work value is KMU
key is: age value is 100
```

# Javascript Variable Scope

- A variable declared with var keyword is scoped
  - In a function, if it is declared in a function
  - Global variable if it is declared outside of a function - a function can access it
  - A same variable name overrides the value of the variable defined outside scope
- A variable without var keyword always becomes a global variable

# Variable Scope Example

- var_scope.js

```
var global_var = 123;
console.log(global_var);

function test1() {
  var local_var = 234;
  console.log(global_var + " " + local_var);
  global_var = 456;
}

test1();
console.log(global_var);

function test2() {
    global_val_function = 789;
}

test2();
console.log(global_val_function);

console.log(local_var);
```

```
123
123 234
456
789
Uncaught ReferenceError: local_var is not defined
```

```
var global_var = 123;

console.log("outer scope: " + global_var);

function scope_test() {
  var global_var = 456;
  console.log("inner scope: " + global_var);
}

scope_test();

console.log("outer scope: " + global_var);
```

```
outer scope: 123
inner scope: 456
outer scope: 123
```

var_same_name.js

# Javascript Prototype

- Every Javascript object inherit properties and methods from a prototype – cf. class inheritance in Java
- A string object inherits all the properties and methods from a String prototype

```
> var str = "sample";
<· undefined
> str.split
    bold
    charAt
    charCodeAt
    codePointAt
    concat
    constructor
    endsWith
    fixed
    fontcolor
    fontsize
    hasOwnProperty
    includes
    indexOf
```

```
propertyIsEnumerable
repeat
replace
search
slice
small
split
startsWith
strike
sub
substr
substring
sup
toLocaleLowerCase
toLocaleString
toLocaleUpperCase
toLowerCase
toString
toUpperCase
trim
trimLeft
trimRight
```

# Access Prototype

- Objects have a prototype where it inheritied from
  - Object create from a literal - var str = “sample”;
  - Object created by using new keywork – var d = new Date();
  - The prototype can be accessed Object.prototype

```
> String.prototype
<· ▼ String {length: 0, [[PrimitiveValue]]: ""} i
    ▶ anchor: function anchor()
    ▶ big: function big()
    ▶ blink: function blink()
    ▶ bold: function bold()
    ▶ charAt: function charAt()
    ▶ charCodeAt: function charCodeAt()
    ▶ codePointAt: function codePointAt()
    ▶ concat: function concat()
```

# How to Create Prototype

- Using an object constructor function

```
function PrototypeName() {
        // properties
        // methods
}
```

# Example of Creating a Prototype

- prototype.js
  - this keyword means the newly created object itself

```
function Person(first, last, gender, country) {
  this.first = first;
  this.last = last;
  this.gender = gender;
  this.country = country;
}
```

```
> var leeky = new Person("kyungyong", "lee", "male", "South Korea");
<· undefined
> leeky
<· Person {first: "kyungyong", last: "lee", gender: "male", country: "South Korea"}
>
```

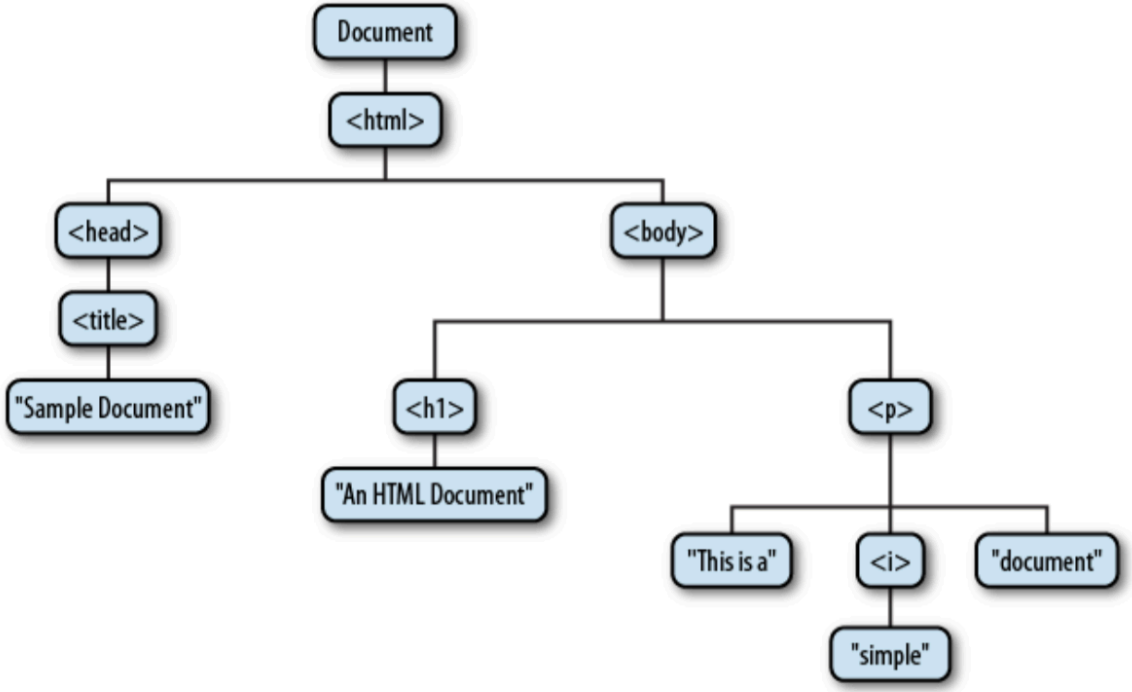
# Window Object in Javascript

- An object to represent open window in a browser
  - A topmost object in a console or page
- You can try window in the console tab
- Handful properties and methods
  - location, location.href, navigator, alert, screen, document, document.body, document.styleSheets (styleSheets[0].cssRules[0].style.cssText)
  - Full list - http://www.w3schools.com/jsref/obj_window.asp

# DOM (Document Object Model)

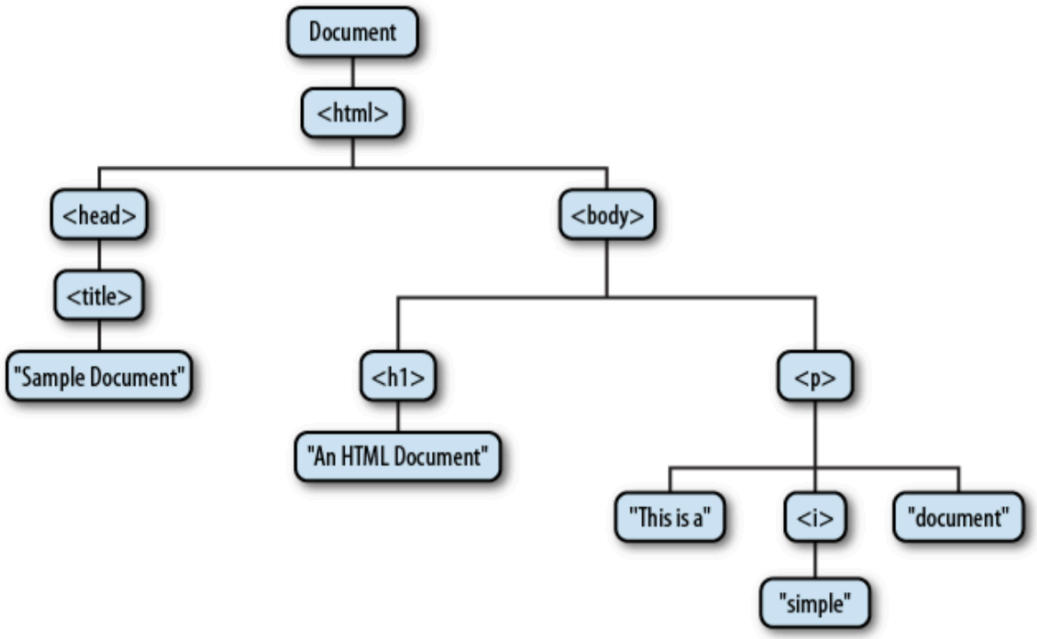
- Represent HTML file as an object so that Javascript can easily access
- Represented in a tree structure where the <html> tag is the root

```
<html>
  <head>
    <title>Sample Document</title>
  </head>
  <body>
    <h1>An HTML Document</h1>
    <p>This is a <i>simple</i> document.
</html>
```



# Element Node in a Tree

- Represents an HTML element
- Parent node
  - A node directly above a node
- Child nodes
  - Nodes one level directly below
- Sibling nodes
  - Nodes at the same level (same parent node)
- Descendant nodes
  - A set of nodes any number of levels below another node
- Ancestor nodes
  - A set of nodes above a node in a tree



# Node in DOM

- Each entry in a tree is a Node
- Element node
  - Representation of element in a HTML <body>, <p>, …
- Attribute node
  - Attribute in an element <a href="">, <img src="">, ...
- Text node
  - Text value within an element
- Example
  - <a href="http://www.kookmin.ac.kr">KMU</a>

Element Attribute Text

# Accessing Element

- document.getElementById(“ID_NAME”)
  - Returns an element with a given ID_NAME
  - Duplicate ID is not allowed in the HTML specification

```
> var t = document.getElementById("first-p");
<· undefined
> t
<· ▼<p id="first-p">
      "The "
      <strong>first</strong>
      " paragraph"
    </p>
> t.innerHTML
<· "The <strong>first</strong> paragraph"
> t.innerText
<· "The first paragraph"
```

```
<!DOCTYPE html>
<html>
<head>
  <title>DOM test</title>
</head>
<body>
  <p id = "first-p">The <strong>first</strong> paragraph</p>
  <p id = "second-p">The second paragraph</p>
  <p id = "first-p">Duplicated ID paragraph</p>
</body>
</html>
```

# Changing Values of an Element

- Assign a value to an element
- Change style of an element
  - Using element.style property

```
> t.innerText = "Changing content";
<· "Changing content"
> t.style.fontSize = "3em";
<· "3em"
> t.style.color="red";
<· "red"
```

Changing content

The second paragraph

Duplicated ID paragraph

# Access Elements By Tag

- document.getElementsByClassName
  - Get a list of elements based on the class name
- document.getElementsByName
  - Get a list of elements based on the name
  - Name is generally used with input tag
- document.getElementsByTagName
  - Get a list of elements with the input tag name

# Accessing Elements Examples

```
> t=document.getElementsByClassName("class-1");
<· [  <p class="class-1">Class 1 paragraph</p>,
    <div class="class-1">Class 1 div</div>]
> t.length
<· 2
> t[0]
<·   <p class="class-1">Class 1 paragraph</p>
> t[1]
<·   <div class="class-1">Class 1 div</div>
```

```
> t=document.getElementsByTagName("p");
<· [  <p class="class-1">Class 1 paragraph</p>,
    <p class="class-2">Class 2 paragraph</p>]
```

```
> t=document.getElementsByName("text-input");
<· [  <input type="text" name="text-input">,
    <input type="text" name="text-input">]
> t[0]
<·   <input type="text" name="text-input">
> t[1].value = "Kyungyong";
<· "Kyungyong"
> t[1].value;
<· "Lee"
```

```
<!DOCTYPE html>
<html>
<head>
  <title>DOM test</title>
</head>
<body>
  <p class = "class-1">Class 1 paragraph</p>
  <p class = "class-2">Class 2 paragraph</p>
  <div class = "class-1">Class 1 div</div>
  ID: <input type="text" name="text-input">
  Name: <input type="text" name="text-input">
</body>
</html>
```

# Adding New Element

- createElement(“element name”);
  - Create a new element to be added in DOM
- createTextNode(“text string for a value”);
  - Create a text node that contains text information
- appendChild(NODE)
  - Append the input node as a child node

# Adding New Element Example

```
> var p_elem = document.createElement("p");
<· undefined
> var content = document.createTextNode("new strings");
<· undefined
> p_elem
<·   <p></p>
> content
<·   "new strings"
> document.body.appendChild(p_elem);
<·   <p></p>
> p_elem.appendChild(content);
<·   "new strings"
```

# Removing Elements

- removeChild(NODE)
  - Remove the input element from the document

```
> document.body.removeChild(p_elem);
<·   <p>new strings</p>
```

# References


- Javascript, the Definitive Guide
- w3schools.com
- tryhelloworld.com